

Demand Forecasting and Recommendations Assignment

- Anish Dhondi

1. SYSTEM DESIGN: BUSINESS KPIS

For our e-commerce platform's ML system to be successful, we need to track specific Key Performance Indicators (KPIs) that directly measure business impact and system performance.

1.1 RECOMMENDATION SYSTEM KPIS

Click-Through Rate (CTR)

- Measures: User clicks on recommended products
- Target: >5% improvement
- Impact: Direct measure of recommendation relevance
- Formula: $(\text{Clicks} \div \text{Recommendations Shown}) \times 100$

Conversion Rate

- Measures: Recommendation clicks leading to purchases
- Target: >3% improvement
- Impact: Direct revenue correlation
- Formula: $(\text{Purchases} \div \text{Clicks}) \times 100$

Customer Retention Rate

- Measures: Customers returning within 30-90 days
- Target: 10-15% improvement
- Impact: Addresses core problem of customer churn
- Formula: $(\text{Returning Customers} \div \text{Total Customers}) \times 100$

Revenue Per User (RPU)

- Measures: Revenue generated per user from recommendations

- Target: 15-20% increase
- Impact: Financial ROI of personalization

Engagement Metrics:

- Session Duration: Average time users spend on platform
- Pages Per Session: Number of product pages viewed per session
- Add-to-Cart Rate: Percentage of recommended products added to cart
- Cross-sell/Up-sell Rate: Success rate of recommending complementary products

1.2 DEMAND FORECASTING KPIS

Mean Absolute Percentage Error (MAPE)

- Measures: Prediction accuracy vs actual demand
- Target: <15% seasonal, <10% regular products
- Impact: Optimizes inventory management
- Formula: $|Actual - Predicted| \div Actual \times 100$

Stockout Rate

- Measures: Time products are out of stock
- Target: <5% normal periods, <10% peak seasons
- Impact: Prevents lost sales and customer dissatisfaction

Inventory Turnover

- Measures: Speed of inventory replacement
- Target: 12-15 times/year for fashion
- Impact: Reduces costs and prevents obsolescence

Operational Metrics:

- Demand Prediction Lead Time: Time between prediction and actual demand
- Forecast Bias: Tendency to over-forecast or under-forecast
- Safety Stock Optimization: Reduction in excess inventory while maintaining service levels

1.3 PLATFORM PERFORMANCE KPIS

System Uptime

- Target: 99.9% during peak events
- Impact: Prevents revenue loss during critical periods

API Response Time

- Target: <200ms recommendations, <500ms search
- Impact: Maintains user experience and prevents abandonment

Scalability Metrics:

- Peak traffic handling capacity
- Auto-scaling efficiency
- Resource utilization optimization

Model Performance

- Model freshness: Weekly updates minimum
- Drift detection: Automated monitoring of performance degradation
- A/B testing: Track improvement rate of new model versions

2. MLOPS ADVANTAGES

MLOps provides critical advantages over basic ML models for e-commerce platforms:

2.1 SCALABILITY AND AUTOMATION

Traditional Model Issues:

- Manual training and deployment
- Limited data processing capacity
- Constant human intervention required
- Poor scalability with business growth

MLOps Solutions:

- End-to-end automation of ML pipeline
- Real-time processing of millions of interactions
- Auto-scaling during peak traffic (Black Friday)
- Continuous learning from new data

2.2 RELIABILITY AND CONSISTENCY

Traditional Model Issues:

- Inconsistent model versioning
- Environment differences between dev/production
- No rollback capabilities
- Limited team collaboration

MLOps Solutions:

- Systematic version control and tracking
- Identical environments across all stages
- One-click rollback to previous versions
- Unified collaboration platform

2.3 MONITORING AND DETECTION

Traditional Model Issues:

- No automated performance monitoring
- Reactive issue identification
- Manual drift detection
- Limited model visibility

MLOps Solutions:

- Real-time accuracy and latency monitoring
- Proactive alerting before customer impact
- Automated data and concept drift detection

- Comprehensive performance dashboards

2.4 BUSINESS VALUE AND RISK

Traditional Model Issues:

- Single points of failure
- Inconsistent performance
- Slow market response
- High maintenance costs

MLOps Benefits:

- Hours vs weeks for model deployment
- Systematic quality assurance
- Automated failover and backup systems
- Reduced operational costs through automation

2.5 GOVERNANCE AND COMPLIANCE

Traditional Model Issues:

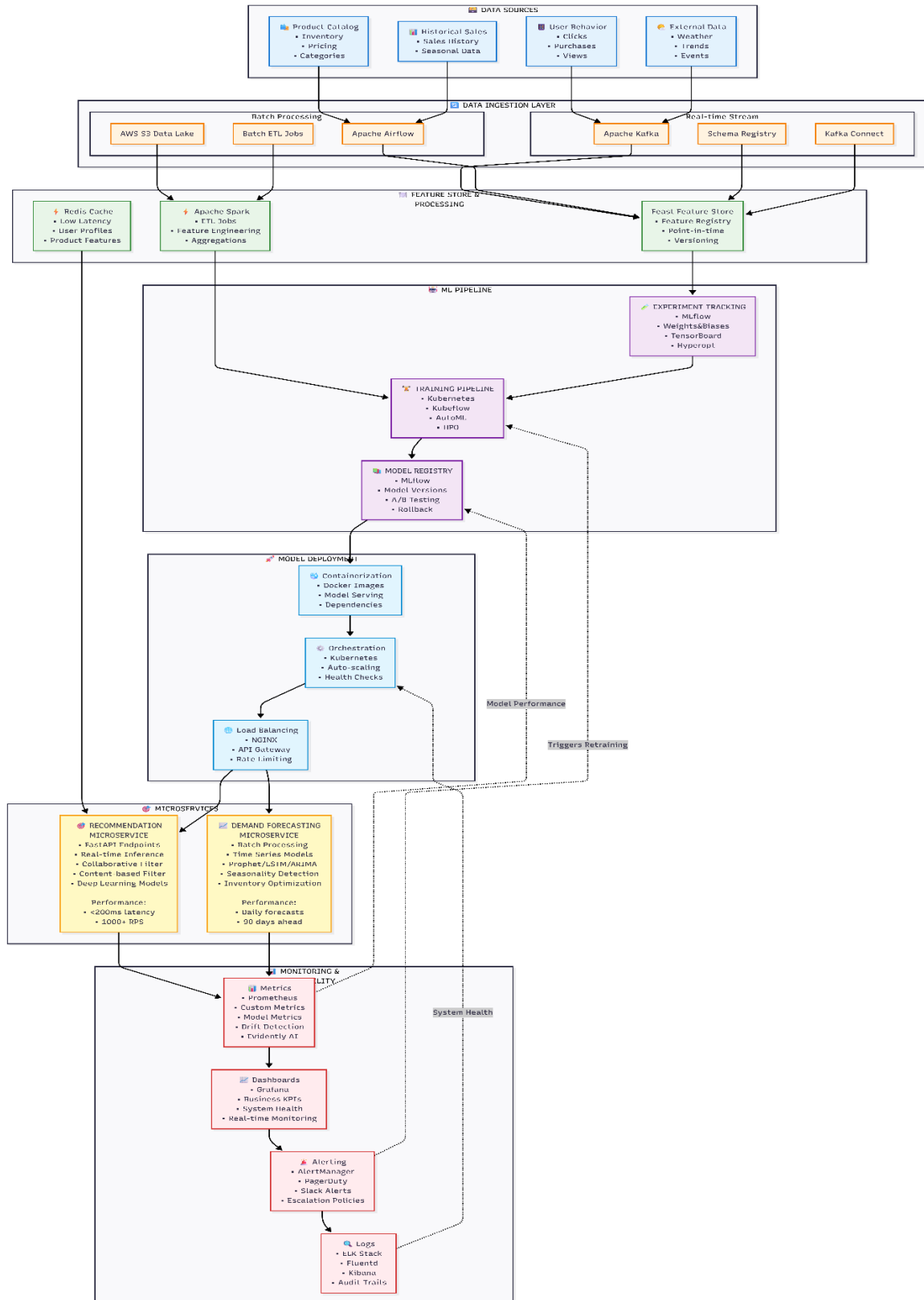
- Isolated team workflows
- No decision audit trails
- Regulatory compliance challenges
- Limited knowledge sharing

MLOps Benefits:

- Cross-functional collaboration platform
- Complete audit trails for all decisions
- Automated compliance documentation
- Centralized knowledge repositories

3. SYSTEM ARCHITECTURE

ARCHITECTURE DIAGRAM:



The MLOps system follows a 7-layer architecture:

3.1 DATA SOURCES LAYER

- User interactions, product catalog, external market data, historical sales

3.2 DATA INGESTION LAYER

- Real-time: Apache Kafka for streaming data
- Batch: Apache Airflow for workflow orchestration

3.3 FEATURE STORE AND PROCESSING LAYER

- Feast Feature Store, Apache Spark, Redis Cache

3.4 ML PIPELINE LAYER

- MLflow for experiment tracking and model registry
- Kubernetes for training pipeline orchestration

3.5 MODEL DEPLOYMENT LAYER

- Docker containerization with Kubernetes orchestration

3.6 MICROSERVICES LAYER

- Recommendation service (real-time, <200ms latency)
- Demand forecasting service (batch processing, 90-day forecasts)

3.7 MONITORING AND OBSERVABILITY LAYER

- Prometheus + Grafana for metrics and visualization
- Alerting systems and centralized logging

COMPONENT CONNECTIONS:

- Data flows from sources → Kafka/Airflow → Feature Store → ML Pipeline → Deployment → Microservices
- Monitoring layer observes all components with feedback loops
- Feature Store provides real-time and batch serving capabilities
- Kubernetes orchestrates training, deployment, and scaling across all layers

4. TOOL SELECTION JUSTIFICATION

The architecture employs a carefully selected combination of open-source and managed services to balance flexibility, cost-effectiveness, and operational excellence.

4.1 DATA INFRASTRUCTURE DECISIONS

Apache Kafka (Real-time Streaming):

- Handles millions of user interactions per second
- Built-in fault tolerance ensuring no data loss during peak traffic
- Horizontal scalability for growing user bases
- Alternative considered: Apache Pulsar

Apache Airflow (Workflow Orchestration):

- Python-native design aligning with ML team expertise
- Clear DAG visualization for complex data pipelines
- Built-in retry logic and error handling mechanisms
- Alternatives considered: Prefect, Kubeflow Pipelines

4.2 MACHINE LEARNING PLATFORM DECISIONS

MLflow (Experiment Tracking & Model Registry):

- Open-source nature avoiding vendor lock-in while remaining cost-effective
- Supports multiple programming languages (Python, R, Java, Scala)
- Standardized model packaging and deployment capabilities
- Alternatives considered: Weights & Biases, Neptune

Kubernetes + Docker (Container Orchestration):

- Automatic resource scaling based on demand
- Built-in redundancy and fault tolerance
- Industry standard with largest ecosystem

- Alternatives considered: AWS ECS, Docker Swarm

4.3 FEATURE STORE SELECTION

Feast Feature Store:

- Open-source architecture providing cost-effectiveness and customization
- Supports both real-time and batch serving patterns
- Ensures point-in-time correctness for consistent training data
- Alternatives considered: Tecton, AWS SageMaker Feature Store

4.4 MONITORING AND OBSERVABILITY DECISIONS

Prometheus + Grafana (Metrics & Visualization):

- Industry standard for metrics and visualization
- Pull-based model operating efficiently at scale
- Built-in alerting through Alert Manager
- Alternatives considered: DataDog, New Relic

Evidently AI (ML-specific Monitoring):

- Purpose-built for machine learning model monitoring
- Advanced data and model drift detection capabilities
- Simple API integration for custom implementations
- Alternatives considered: Arize, WhyLabs

4.5 HYBRID APPROACH RATIONALE

Strategic Benefits:

- Core infrastructure uses open-source tools (Kafka, Kubernetes, MLflow) for flexibility
- Managed storage services (AWS RDS, S3) provide reliability and reduced operational overhead
- Monitoring combines open-source solutions with managed alerting services

- Cost optimization through selective use of managed services
- Maintains vendor flexibility for future requirements
- Ensures operational excellence through proven enterprise-grade tools
- Scales effectively with business growth while maintaining technical sophistication

5. SOLUTION WORKFLOW

The workflow outlines the end-to-end process of building and operating the MLOps system for demand forecasting and recommendations.

5.1 DATA AND MODEL EXPERIMENTATION

Tools Used: Jupyter Hub, MLflow, Git, DVC, Feast Feature Store

Workflow Steps:

- Sandbox Environment: Set up isolated experimentation environment using Kubernetes namespaces
- Data Access: Connect to Feature Store (Feast) for standardized data access and exploration
- Experiment Tracking: Configure MLflow tracking server for systematic experiment management
- Model Development: Test collaborative filtering, content-based filtering, and hybrid approaches for recommendations
- Demand Forecasting: Experiment with ARIMA, Prophet, LSTM, and ensemble methods
- Evaluation Criteria: Use Precision@K, Recall@K, NDCG for recommendations; MAPE, RMSE, MAE for forecasting
- Model Comparison: Use MLflow UI for experiment comparison and statistical significance testing

5.2 AUTOMATION OF DATA PIPELINE

Real-time Data Ingestion:

- Apache Kafka: Captures user interaction events (clicks, purchases, cart additions)
- Schema Registry: Ensures data consistency and validation
- Kafka Connect: Seamless integration with various data sources

Batch Data Processing:

- Apache Airflow: Orchestrates daily data workflows with DAG visualization
- Apache Spark: Handles large-scale feature engineering and ETL operations
- AWS S3: Provides centralized data lake storage solution

Feature Engineering Pipeline:

- User Features: Purchase history, browsing patterns, demographics, seasonality preferences
- Product Features: Category, price range, popularity scores, seasonal trends
- Contextual Features: Time of day, weather, ongoing promotions
- Quality Assurance: Data validation using Great Expectations and automated alerts

5.3 AUTOMATION OF TRAINING PIPELINE

Tools Used: Kubeflow Pipelines, MLflow, Kubernetes, Optuna

Training Pipeline Steps:

- Data Validation: Automated data quality checks before training
- Feature Engineering: Standardized feature preparation and selection
- Model Training: Distributed training using Kubernetes and Kubeflow
- Hyperparameter Optimization: Automated tuning using Optuna integrated with MLflow
- Model Validation: Cross-validation, business logic testing, and A/B testing framework
- Performance Benchmarking: Latency and throughput testing
- Model Registration: Centralized model versioning in MLflow registry
- Automated Deployment: Blue-green and canary deployment strategies with automated rollback

5.4 AUTOMATION OF INFERENCE PIPELINE

Real-time Recommendation Serving:

- FastAPI: RESTful API endpoints for recommendation requests
- Redis Cache: Low-latency access to user profiles and popular items

- NGINX Load Balancer: High availability and traffic distribution
- Kubernetes HPA: Auto-scaling based on CPU/memory usage
- Performance Target: <200ms latency, 1000+ requests per second

Batch Demand Forecasting:

- Daily Processing: Automated forecasting pipeline running at 2 AM daily
- Time Series Models: Prophet, LSTM, ARIMA for different product categories
- Seasonality Detection: Automatic identification of seasonal patterns
- Inventory Integration: Direct updates to inventory management system
- Forecast Horizon: 90-day ahead predictions with daily updates

5.5 CONTINUOUS MONITORING PIPELINE

Performance Monitoring:

- Prometheus: Metrics collection for API latency, throughput, error rates
- Grafana: Real-time dashboards for business KPIs and system health
- Custom Metrics: Model accuracy, conversion rates, recommendation relevance

Data and Model Drift Detection:

- Evidently AI: Statistical tests for data distribution changes
- Automated Alerts: Slack/email notifications when drift thresholds exceeded
- Feature-level Analysis: Identification of specific drifting features
- Performance Tracking: Continuous monitoring of model accuracy trends

Business Impact Monitoring:

- Real-time KPI Tracking: CTR, conversion rate, revenue attribution
- A/B Testing Framework: Continuous model performance comparison
- Alert Categories: Critical (immediate response), Warning (1-hour response), Info (notification only)

6. SPECIAL SCENARIOS

6.1 DRIFT DETECTION HANDLING

Drift Detection Components:

- Evidently AI: Statistical drift detection using KL-divergence and Kolmogorov-Smirnov tests
- Threshold Setting: Configurable drift thresholds based on business impact
- Feature-level Monitoring: Individual feature drift tracking and analysis
- Automated Logging: Complete drift detection event recording

When Drift is Detected (Below Threshold):

- Warning Logs: Record drift detection event with severity level
- Enhanced Monitoring: Increase data collection frequency and analysis depth
- Feature Analysis: Identify specific features contributing to drift
- Stakeholder Notification: Alert data science team via Slack/email integration

When Drift Exceeds Threshold:

- Immediate Actions: Trigger automated model performance evaluation
- Root Cause Analysis: Analyze drift patterns and identify potential causes
- Model Rollback: Automatic reversion to previous stable model version
- Fallback Strategies: Activate rule-based recommendation system as backup
- Emergency Response: Page on-call engineering team for immediate intervention

Automated Response Workflow:

- Performance Check: Validate current model accuracy against baseline
- Data Pipeline Audit: Verify data source integrity and processing pipeline
- Model Retraining: Trigger automated retraining with recent data
- Stakeholder Communication: Send detailed drift analysis reports
- Recovery Monitoring: Track system recovery and performance restoration

6.2 NEW ANNOTATED DATA HANDLING

Data Ingestion Triggers:

- User Feedback: Thumbs up/down on recommendations, rating systems
- Purchase Confirmations: Implicit positive feedback from completed purchases
- Customer Service: Interaction logs and review sentiment analysis
- A/B Test Results: Labeled outcomes from controlled experiments

Automated Processing Pipeline:

- Data Validation: Quality checks for new annotation consistency
- Volume Assessment: Determine if data volume justifies retraining
- Performance Impact: Evaluate potential improvement from new annotations
- Training Strategy Selection: Choose between full retraining vs incremental learning

Incremental Learning Strategy:

- Online Learning: Real-time model updates for recommendation engines
- Batch Updates: Weekly full retraining for demand forecasting models
- Hybrid Approach: Combine online updates with periodic full retraining
- Performance Monitoring: Track improvement metrics after incorporating new data

Quality Assurance for Annotations:

- Consistency Checks: Validate annotation quality and remove outliers
- Bias Detection: Monitor for annotation bias and data imbalance
- Ground Truth Validation: Cross-reference with historical performance data
- Feedback Loop: Incorporate annotation quality metrics into model training

Automated Retraining Triggers:

- Volume Threshold: Retrain when annotation count exceeds defined threshold
- Performance Degradation: Trigger retraining when accuracy drops below target
- Time-based Schedule: Regular retraining cycles regardless of data volume
- Business Events: Retrain after major product launches or seasonal changes

Decision Matrix for Retraining:

- Immediate Retraining: High-value annotations, significant performance drop
- Scheduled Retraining: Moderate annotation volume, stable performance
- Hold and Monitor: Low annotation volume, acceptable current performance
- Emergency Retraining: Critical system issues, major data distribution changes